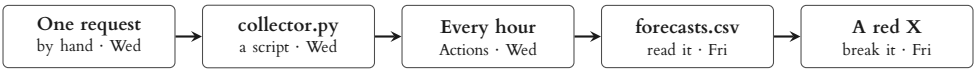
Automating Data Collection

Week 14 · Practice module · Textbook Chapter 14

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

November 18 & 20, 2026

This week at a glance



Monday, Nov 16 | Week 13's lab, Part 3
Its frames are in week 13's deck.

Wednesday, Nov 18 | Part 1
One request by hand, then your own collector running in GitHub Actions.

Friday, Nov 20 | Part 2
What it gathered, failures and monitoring. Then 15 minutes on portfolio checkpoint 4.

Bring

Your GitHub account, and `boulder-forecast-template.zip` from Canvas. No key.

Due

Sunday, Nov 22: chapter 14's Recommended Exercises.
Sunday, Nov 29: portfolio checkpoint 4.

1. Wednesday · Live-Coding Lab

One request is a snapshot

Forecasters change Boulder's forecast a few times a day. On November 12, one forecast stood for 12 hours. One request tells you what it said once.

To see it change, a program has to ask again and again, on a schedule.

What else from this semester would you want to watch change?

This week's source

The National Weather Service's hourly forecast for Boulder. No key.

This week's lab

Find it | `labs/week-14` in the course repository

1. Open `lab-14.ipynb` and click **Download raw file**.
2. Open it in JupyterLab, in the `webdata` environment.
3. Sign in to GitHub in your browser.

Use it | one part a day

- I type the code. You type it in your copy.

Fall behind?

Copy the cell from `lab-14-key.ipynb`, beside it, and keep going.

Step 0: set up

This cell is given. Put your email in `HEADERS`, and run it.

The NWS asks for no key. It asks every program to name itself and give a contact. Why send your email if nobody checks?

Docs: the NWS's API documentation, Overview tab

```
# The two libraries we use
import requests
import pandas as pd
# Name our program, and give your email
HEADERS = {"User-Agent":
           "WebDataScience/1.0 (your-email@colorado.edu)"}
```

Step 0: two addresses

The same cell names two addresses. `nws_base` is the NWS's API. `template` is where the course keeps `collector.py` and its workflow.

Steps 3 and 4 print those two files from it.

```
# The NWS's API
nws_base = "https://api.weather.gov"
# Where the course keeps this week's template
template = (
    "https://raw.githubusercontent.com/"
    "cuinfoscience/INFO4617-Fall2026/main/"
    "labs/week-14/template/"
)
```

Step 1: where is Boulder?

The NWS forecasts for a grid of squares about 2.5 km on a side.

`/points` takes a latitude, then a longitude, and names the square. West is negative. Which number is the latitude?

```
# Boulder: the latitude, then the longitude
points_url = f"{nws_base}/points/40.015,-105.2705"
```

Step 1: send it

Send it with `HEADERS`. Print the status code and the `Content-Type` header.

Is the answer JSON? Which kind?

```
# Send it, with our headers
points_response = requests.get(
    points_url,
    headers=HEADERS,
)
# The status code, and the kind of data
print(
    points_response.status_code,
    points_response.headers["Content-Type"],
)
```

Step 1: any error?

When something goes wrong, the NWS says what in `detail`. `None` means no error.

Bluesky's errors had a `message`. Why read the error before the data?

```
# The answer, as a dictionary
points = points_response.json()
# Any error? None means no error
print(points.get("detail"))
```

Step 1: one level at a time

Print the answer's keys. Then print the keys of its `properties`.

Which key looks like the address of an hourly forecast?

```
# The answer's keys
print(points.keys())
# The keys one level down, in "properties"
print(points["properties"].keys())
```

Step 1: the square

Each square belongs to one of the NWS's forecast offices.

Which office, which square, and which city did you get?

```
# The part of the answer about the square
grid = points["properties"]
# Its office, and the square's two numbers
print(
    grid["gridId"],
    grid["gridX"],
    grid["gridY"],
)
# The city nearest the square
print(grid["relativeLocation"]["properties"]["city"])
```

Step 1: the forecast's address

`forecastHourly` is the address our collector will ask every hour.

The documentation says a square can change. `collector.py` keeps only this address. What would it collect if the square changed?

```
# The address of the hourly forecast
hourly_url = grid["forecastHourly"]
print(hourly_url)
```

Microsprint: the numbers swapped

The likeliest slip is the order of the two numbers. Ask with the longitude first.

What status code came back? What did the NWS say was wrong, in its own words?

```
# The longitude first, by mistake
swapped_response = requests.get(
    f"{nws_base}/points/-105.2705,40.015",
    headers=HEADERS,
)
# The answer, as a dictionary
swapped = swapped_response.json()
# The status code, and the NWS's error
print(swapped_response.status_code, swapped.get("detail"))
# Which part of the request was wrong
print(swapped.get("parameterErrors"))
```

Step 2: the hourly forecast

Ask `hourly_url`, the same way.

Is the status code the same as for `/points`?

```
# Send it, the same way
hourly_response = requests.get(
    hourly_url,
    headers=HEADERS,
)
# The status code, and the kind of data
print(
    hourly_response.status_code,
    hourly_response.headers["Content-Type"],
)
```

Step 2: one period per hour

Its `properties` hold a list of `periods`, one for each hour ahead.

How many periods? How many days is that?

```
# The answer, as a dictionary
hourly_json = hourly_response.json()
# The part that holds the forecast
hourly = hourly_json["properties"]
# How many periods it holds
print(len(hourly["periods"]), "periods")
```

Step 2: one hour's forecast

Print the first period.

Which of its fields would you save every hour? Which would you leave out?

```
# The first period: one hour's forecast
print(hourly["periods"][0])
```

Step 2: how new is it?

`updateTime`: when forecasters last changed the forecast.

`generatedAt`: when the NWS's server built this answer.

Which is older? By how much?

```
# When forecasters last changed the forecast
print("updated: ", hourly["updateTime"])
# When the NWS's server built this answer
print("generated:", hourly["generatedAt"])
```

Step 2: what the headers say

`Last-Modified` dates this answer. `Cache-Control` says how long a copy may be reused: `max-age` by your browser, `s-maxage` by the NWS's caches.

Thirty of us just asked. How many times did the NWS build it?

Docs: MDN's Cache-Control

```
# When this answer was last changed
print(
    "Last-Modified:",
    hourly_response.headers["Last-Modified"],
)
# How long a copy of it may be reused
print(
    "Cache-Control:",
    hourly_response.headers["Cache-Control"],
)
```

Microsprint: how old is the forecast?

`pd.to_datetime()` turns each time into one Python can subtract.

How old was your forecast when the NWS built your answer? And your neighbor's?

```
# Each time, as one Python can subtract
generated = pd.to_datetime(hourly["generatedAt"])
updated = pd.to_datetime(hourly["updateTime"])
# How old the forecast was when the answer was built
print(generated - updated)
```

`collector.py` is the cells above, saved as a script that a computer can run without us. A given cell prints it.

Find our request in it. Then find four things a notebook didn't need.

The four

Constants. Append mode. `raise_for_status()`. The `__main__` guard.

collector.py: constants at the top

Each setting is written once, at the top, and changed in one place.

Which line would you change to collect another city's forecast?

```
# The hourly forecast for Boulder's grid square
URL = (
    "https://api.weather.gov/gridpoints/BOU/54,75/"
    "forecast/hourly"
)
# Name our program, and give your email
HEADERS = {"User-Agent":
           "WebDataScience/1.0 (your-email@colorado.edu)"}
```

collector.py: the file, its columns, a last day

`FIELDS` names each row's columns, in order. The `...` stands for four more.

Which column would you add? From which field of a period?

```
# Where the rows go
FILE = "forecasts.csv"
# The columns each row has, in order
FIELDS = [
    "collected_at",
    "update_time",
    "start_time",
    "temperature",
    ... # four more
]
# The last day to collect
LAST_DAY = date(2026, 12, 11)
```

collector.py: stop after the last day

After December 11, `main()` prints a line and stops before it asks.

Why would a collector need a last day? Who would a forgotten one keep asking?

```
def main():
    # After the last day, stop before asking
    if datetime.now(timezone.utc).date() > LAST_DAY:
        print("Collection ended on", LAST_DAY)
        return
    ...
```

collector.py: fail where someone will see

`raise_for_status()` stops the script with an error if the answer isn't 200.

Our notebook cells never use it. Why does a script that runs alone need it?

Docs: raise_for_status()

```
# Ask, and give up after 30 seconds
response = requests.get(
    URL,
    headers=HEADERS,
    timeout=30,
)
# Stop with an error if the answer isn't 200
response.raise_for_status()
# The part that holds the forecast
forecast = response.json()["properties"]
```

collector.py: when we asked

Every row will repeat when we asked and when the NWS last updated the forecast.

Why keep those two on every row?

`collector.py` gets `collected_at` in one line.

```
# The time now, in UTC
now = datetime.now(timezone.utc)
# As text, to the second
collected_at = now.isoformat(timespec="seconds")
# When the NWS last updated the forecast
update_time = forecast["updateTime"]
```

collector.py: append

`"a"` opens the file to add rows at its end.

What would `"w"` do to last week's rows?

```
# Is this the first run? Then there's no file yet
new_file = not os.path.exists(FILE)
# Open the file to add rows at its end
with open(
    FILE,
    "a",
    newline="",
    encoding="utf-8",
) as f:
    ... # write the rows: the next two frames
```

collector.py: write the header once

The header goes in only when the file is new.

`new_file` is set before `open()`. Why not after?

Docs: csv.DictWriter

```
with open(...) as f:
    # A writer that turns a dictionary into a row
    writer = csv.DictWriter(f, fieldnames=FIELDS)
    # The header, only in a new file
    if new_file:
        writer.writeheader()
    # Then a row for each hour
    for period in forecast["periods"]:
        ...
```

collector.py: one row per hour

Each period becomes one row: a dictionary whose keys are the names in `FIELDS`.

What would `writerow()` do with a key that isn't in `FIELDS`?

```
for period in forecast["periods"]:
    # One row per hour, as a dictionary
    writer.writerow({
        "collected_at": collected_at,
        "update_time": update_time,
        "start_time": period["startTime"],
        "temperature": period["temperature"],
        ... # four more fields
    })
```

collector.py: run as a script

The last two lines run `main()` only when the file is run as a script, as the workflow does with `python collector.py`.

Where else in the file would you look for a bug?

Docs: `__main__`

```
# Run main() only when the file is run as a script
if __name__ == "__main__":
    main()
```

Step 4: your own repository

1. Unzip `boulder-forecast-template.zip`. It is in this week's Canvas module.
2. On GitHub, click **+** at the top right. Then click **New repository**.
   - Type the name `boulder-forecast`. Select **Public**.
   - Turn on the README option. Click **Create repository**.

Why a README?

It gives the new repository its `main` branch.

Step 4: upload two files

3. Click **Add file**, then **Upload files**. Drag in `collector.py` and `README.md`. Click **Commit changes**.
4. Click `collector.py`, then the pencil. In `HEADERS`, type your email address. Commit the change.

No `forecasts.csv`?
The first run makes it.

Step 4: create the workflow

5. Click **Add file**, then **Create new file**.
   - Type the name `.github/workflows/collect.yml`. Each `/` makes a folder.
   - Run the given cell under Step 4. Copy all that it prints into the big box.
   - Commit the change.

Why not upload it?

Its folder's name starts with a dot. macOS hides it.

collect.yml: when it runs

A **workflow** is a file of steps. GitHub runs it on its own computers when something happens.

Here, two things: a `schedule`, and a button, `workflow_dispatch`. When will yours run first?

Docs: schedule

```
# The name on the Actions tab
name: Collect Boulder's hourly forecast

# When it runs:
on:
  # at minute 17 of every hour, in UTC
  schedule:
    - cron: "17 * * * *"
  # and when you click "Run workflow"
  workflow_dispatch:
```

Five fields, in UTC

`cron` is the old Unix scheduler. Actions reads its five fields, in UTC unless told otherwise. A `*` means every value.

Runs at minute 0 are delayed most often. Which Boulder hour is 13:30 UTC?

Try one: crontab.guru

min	hour	day	month	weekday	**runs at**
`17`	`*`	`*`	`*`	`*`	minute 17 of every hour
`0`	`*/6`	`*`	`*`	`*`	every six hours
`30`	`13`	`*`	`*`	`1-5`	13:30, Monday to Friday

collect.yml: permission to commit

`contents: write` lets the workflow commit to your repository.

Why does it need that? What else could it do with it?

Docs: permissions

```
# Let the workflow commit to this repository
permissions:
  contents: write

# One job, on a fresh Ubuntu machine
jobs:
  collect:
    runs-on: ubuntu-latest
    # Its steps: the next three frames
    steps:
      ...
```

collect.yml: a fresh computer each time

Each run starts on an empty Ubuntu machine. First it copies your repository's files. Then it installs Python.

What would be missing if a step were left out?

```
steps:
  # Copy the repository onto the empty machine
  - name: Get this repository's files
    uses: actions/checkout@v7

  # Install Python, version 3.12
  - name: Set up Python
    uses: actions/setup-python@v7
    with:
      python-version: "3.12"
```

collect.yml: run the collector

These two steps are what you would type in a terminal.

Step names show in the run's log. Which step will you open when a run fails?

```
# Install the one library collector.py needs
- name: Install requests
  run: pip install requests

# Then run our script
- name: Collect the forecast
  run: python collector.py
```

collect.yml: commit the new rows

The last step commits `forecasts.csv` as `github-actions[bot]`, if any rows changed.

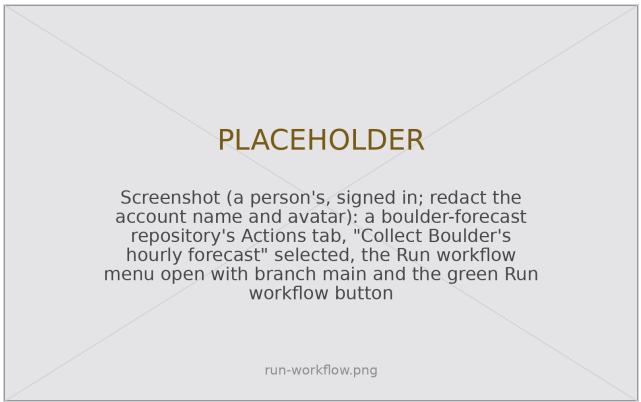
If the collector fails, this step is skipped. What does your file look like then?

```
# Commit the CSV, if any rows changed
- name: Commit the new rows
  run: |
    # Sign the commit as GitHub's bot
    git config user.name "github-actions[bot]"
    git config user.email "..."
    # Stage the CSV; commit only if it changed
    git add forecasts.csv
    git diff --staged --quiet || git commit -m "..."
    # Take any newer commits, then push
    git pull --rebase
    git push
```

Step 5: start the first run

A scheduled run can start late. Start the first one by hand.

6. Click the **Actions** tab. If GitHub asks, enable workflows. Click **Collect Boulder's hourly forecast**. Click **Run workflow**, then the green **Run workflow**.



Step 5: read the log

7. Refresh the page until the run shows. Click the run, then the job `collect`. Click **Collect the forecast** to open that step.

How many hours did it save? Which `updateTime`?

A green check

Every step worked.

Docs: workflow run logs

Step 5: see the commit

8. Click the **Code** tab. Find `forecasts.csv`, and the commit by `github-actions[bot]`. Click the file.

From now on it runs by itself, every hour, until December 11.

Git scraping

Each run kept as a commit, as newsrooms do. What can the history show that the file can't?

Step 6: your raw address

GitHub serves each file of a public repository at a **raw** address: the file alone, without GitHub's page.

Put your user name in place of `<you>`.

```
# Your file's raw address, with your user name
my_series = (
    "https://raw.githubusercontent.com/"
    "<you>/boulder-forecast/main/forecasts.csv"
)
```

Step 6: read what your collector saved

`pd.read_csv()` reads from an address too. The raw copy can be five minutes behind.

Which columns would differ from your neighbor's file?

```
# Read the CSV straight from its address
my_forecasts = pd.read_csv(my_series)
# How many rows and columns
print(my_forecasts.shape)
# Its first rows
my_forecasts.head()
```

Open your **Actions** tab once. Each run is a line, with a green check or a red X.

How many runs do you expect by Friday's class?

Friday

Part 2 starts at Step 7, with its own setup. Today's cells don't need to have run.

2. Friday · Live-Coding Lab

Today: Part 2, what the collector gathered

The course has run the same `collector.py` every hour since November 13.

Today we count how often the forecast changed and follow one hour as it came closer. Then we break a run on purpose.

Start at Step 7

Download the new `lab-14.ipynb`. Missed Wednesday? Start here.

Step 7: set up again

This cell is given. Put your email in `HEADERS`, and run it.

`matplotlib` is new today. Step 9 draws a plot with it.

```
# The libraries for Part 2
import requests
import pandas as pd
import matplotlib.pyplot as plt
# Name our program, and give your email
HEADERS = {"User-Agent":
           "WebDataScience/1.0 (your-email@colorado.edu)"}
```

Step 7: two addresses

The same cell names two addresses.

`series` is the course's file. A new run adds to it at minute 17 of every hour.

```
# The NWS's API
nws_base = "https://api.weather.gov"
# The course's series, collected every hour
series = (
    "https://raw.githubusercontent.com/"
    "cuinfoscience/INFO4617-Fall2026/main/"
    "labs/week-14/data/forecasts.csv"
)
```

Step 7: the course's series

Read it, and look at its last rows.

What can this file tell you that a live request can't? What can't it tell you?

```
# Read the course's CSV
forecasts = pd.read_csv(series)
# How many rows and columns
print(forecasts.shape)
# Its last rows
forecasts.tail()
```

Step 8: runs, and forecasts

Each run is one `collected_at`. Each forecast the NWS issued is one `update_time`.

If the forecast changed every hour, the two counts would match. Do they?

```
# How many runs: distinct collected_at times
print(forecasts["collected_at"].nunique(), "runs")
# How many forecasts: distinct update_time values
print(forecasts["update_time"].nunique(), "forecasts")
```

Step 8: each forecast, and its runs

This cell is given. For each forecast, it counts the runs that collected it, and finds the first and last.

Which forecasts lasted longest? What does that say about when forecasters update it?

The notebook groups and counts in one line.

```
# The run times, grouped by forecast
runs = forecasts.groupby("update_time")["collected_at"]
# For each: how many runs, the first, and the last
per_forecast = runs.agg([
    "nunique",
    "min",
    "max",
])
# Print every row of the table
print(per_forecast.to_string())
```

Microsprint: the share that was new

Divide forecasts by runs: the share of runs that brought a forecast no earlier run had.

Thirty collectors each ask 24 times a day. How many of their 720 requests bring something new?

The notebook divides in one line.

```
# How many forecasts, and how many runs
n_forecasts = forecasts["update_time"].nunique()
n_runs = forecasts["collected_at"].nunique()
# The share of runs that brought a new forecast
new_share = n_forecasts / n_runs
print(round(new_share, 2))
# The same share of 720 requests a day
print(round(720 * new_share), "of 720 daily requests")
```

What a bot costs a server

Step 8 counted how often the forecast changed. Our collector asks 24 times a day anyway. On November 12, an hourly collector got the same forecast 12 times in a row.

`Last-Modified` and `Cache-Control` say how new an answer is. Who pays for the requests that bring nothing new?

Ask less?

What would a collector need to know to ask less often and miss nothing?

Step 9: one hour, forecast again and again

Every run forecasts the next six and a half days. So each hour is forecast by many runs.

Take noon on Wednesday, our first class. How many runs forecast it?

The notebook picks these rows in one line.

```
# Noon on Wednesday, November 18, in Boulder's time
noon = "2026-11-18T12:00:00-07:00"
# True for each row that forecasts that hour
is_noon = forecasts["start_time"] == noon
# Keep those rows, and count them
noon_rows = forecasts[is_noon]
print(len(noon_rows))
```

Step 9: lead time

A row's **lead time** is how far ahead its run was: the hour's `start_time` minus the run's `collected_at`.

This cell is given. The notebook subtracts in one line, without naming `hour` and `run`.

```
# Noon, and when each run asked, as times in UTC
hour = pd.to_datetime(noon_rows["start_time"], utc=True)
run = pd.to_datetime(noon_rows["collected_at"], utc=True)
# Lead time: how far ahead each run was
lead = hour - run
# The same, in hours
lead_hours = lead.dt.total_seconds() / 3600
```

Step 9: plot it

The same given cell plots the forecast temperature against the lead time.

Where does the line settle?

```
# One figure, wide and short
fig, ax = plt.subplots(figsize=(8, 4))
# Temperature against lead time, a dot per run
ax.plot(
    lead_hours,
    noon_rows["temperature"],
    marker=".",
)
# Furthest ahead on the left, nearest on the right
ax.invert_xaxis()
# Then a title, axis labels, and plt.show()
...
```

Step 9: furthest ahead, and nearest

Print the first row and the last: the furthest ahead and the nearest.

Which one was right? What else would you have to collect to know?

The notebook writes the list of columns into each line.

```
# The three columns to show
columns = [
    "collected_at",
    "temperature",
    "short_forecast",
]
# The first run, furthest ahead
print(noon_rows[columns].head(1))
# The last run, nearest
print(noon_rows[columns].tail(1))
```

Microsprint: midnight

Do the same for midnight that night.

How far apart were its lowest and highest forecasts? Is that more or less than noon's?

The notebook picks the rows in one line.

```
# Midnight's rows, picked as for noon, and how many
midnight = "2026-11-19T00:00:00-07:00"
is_midnight = forecasts["start_time"] == midnight
midnight_rows = forecasts[is_midnight]
print(len(midnight_rows))
# Its lowest and highest forecast temperature
print(
    midnight_rows["temperature"].min(),
    midnight_rows["temperature"].max(),
)
```

Step 10: is it still running?

A collector that stops doesn't say so. Its file just stops growing.

A **freshness check** asks how old the newest row is. Under about an hour and a half is fresh. Is the course's?

```
# The newest run's time
newest = pd.to_datetime(forecasts["collected_at"]).max()
# That time, and how long ago it was
print(newest, pd.Timestamp.now(tz="UTC") - newest)
```

Step 10: is yours?

Check your own file the same way, from Wednesday's raw address.

Is it fresh? If not, where would you look first?

The notebook reads the file and finds its newest run in one line.

```
# Your raw address, as on Wednesday
my_series = (
    "https://raw.githubusercontent.com/"
    "<you>/boulder-forecast/main/forecasts.csv"
)
# Read your file
my_rows = pd.read_csv(my_series)
# Its newest run, and how long ago it was
my_newest = pd.to_datetime(my_rows["collected_at"]).max()
print(my_newest, pd.Timestamp.now(tz="UTC") - my_newest)
```

Step 10: is the forecast fresh?

Rows can keep coming while `update_time` stands still. Half a day without a change can be normal.

How long ago did the forecasters last change it? How old would you call stale? Where could this check run to warn you?

The notebook prints this in one line.

```
# When the forecasters last changed the forecast
updated = pd.to_datetime(forecasts["update_time"]).max()
# How long ago that was
print(pd.Timestamp.now(tz="UTC") - updated)
```

Step 11: a mistyped address

This request is **designed to break**. A deliberate error needs no GitHub issue; only code that is broken by accident does.

Where is the typo?

```
# The hourly forecast's address, mistyped
broken_url = (
    f"{nws_base}/gridpoints/BOU/54,75/"
    "forecast/hourlyy"
)
# Send it
broken_response = requests.get(
    broken_url,
    headers=HEADERS,
)
```

Step 11: what came back

Read the answer, and print its status code and `detail`.

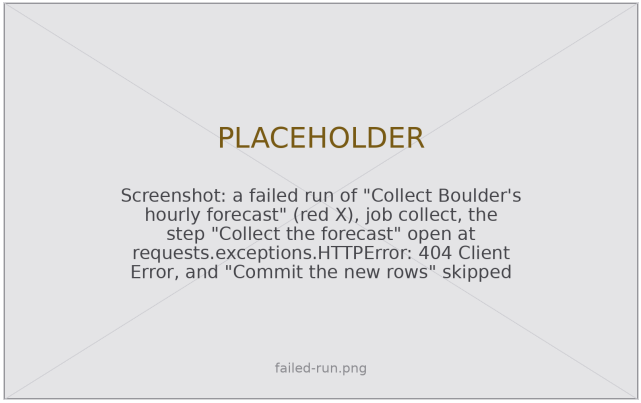
What status code? Does the `detail` say what was wrong?

```
# The answer, as a dictionary
broken = broken_response.json()
# The status code, and the NWS's error
print(broken_response.status_code, broken.get("detail"))
```

Step 11: break your collector

1. Edit `collector.py`. In `URL`, change `hourly` to `hourlyy`. Commit the change.
2. Run the workflow. Open the run with the red X, then its job.

Which line of the log names the error? Was the commit step run?



Step 11: fix it

3. Change `hourlyy` back to `hourly`. Commit the change. Run the workflow again, and wait for the green check.

A red X is a loud failure: GitHub can email whoever made the workflow. Did you get an email?

Docs: notifications for workflow runs

Microsprint: not every mistype fails

Ask for the hourly forecast with the grid numbers swapped, `75,54`.

Do you expect an error?

```
# The grid numbers swapped
swapped_grid_url = (
    f"{nws_base}/gridpoints/BOU/75,54/"
    "forecast/hourly"
)
# Send it
swapped_grid_response = requests.get(
    swapped_grid_url,
    headers=HEADERS,
)
```

Microsprint: what came back

Print the status code and the first hour's temperature.

What status code? Did anything tell you it wasn't Boulder?

The notebook prints the temperature without naming it first.

```
# The part that holds the forecast
swapped_grid = swapped_grid_response.json()["properties"]
# The first hour's temperature
temperature = swapped_grid["periods"][0]["temperature"]
# The status code, and that temperature
print(swapped_grid_response.status_code, temperature)
```

Quiet failures

Some failures leave no red X. A square near Aurora answers 200. A forecast can stop changing. GitHub pauses a public repository's schedule after 60 days without activity.

Which would you rather find a month from now: a gap, or a month of Aurora's forecasts?



Other places to run a schedule

Actions is one of several hosted schedulers. Each free tier has its own limit. Which of these could run our collector every hour, for free?

Service	What runs	Free tier, read on Nov 13, 2026
GitHub Actions (ours)	A workflow's steps	Free in public repositories; every 5 minutes at most
GitLab scheduled pipelines	A CI/CD pipeline	400 compute minutes a month; 24 runs a day
PythonAnywhere scheduled tasks	A Python script	None on free accounts made since January 2026
Cloudflare Workers Cron Triggers	A Worker, in JavaScript or Python	5 cron triggers; 10 ms of CPU a run
Google Cloud Scheduler	A call to a URL you give it	3 jobs a month per billing account
Val Town	A val, in TypeScript	Every 15 minutes at most

cron on your own computer

`crontab -e` opens your computer's schedule. This line runs the collector at minute 17, in your computer's time zone.

It runs only while the computer is awake. What happens when your laptop sleeps?

```
# At minute 17 of every hour, run the collector
17 * * * * cd ~/boulder-forecast && python3 collector.py
```

Wrong Python? Give the full path that `which python3` prints in `webdata`.

Step 12: what did we learn?

Answer in the notebook, as comments.

- How often should a collector ask?
- How far did noon's forecast move?
- Name a failure with a red X, and one without.
- What should your README say about when you collected?

Due Sunday, Nov 22

Chapter 14's Recommended Exercises, from its companion notebook. Graded on a good-faith attempt.

Friday's last 15 minutes: portfolio checkpoint 4

Due Sunday, Nov 29 | Draft your write-up

- **A full draft** of `report.md` or `report.pdf`: every section, your own results and figures, your sources cited
- **All four files**, present and current: `README.md`, `proposal.md`, your notebook, and your report
- **A README** that lets someone else reproduce the project: setup, how to run it, where the data came from

Work on your README's **How to reproduce** now: when you sent your requests, and from what code.
Next: final presentations, after Fall Break.

Key takeaways

1. One request is a **snapshot**. A series needs a program on a schedule.
2. A collector **appends**, and records when it asked.
3. **Count** what changed before deciding how often to ask.
4. A red X is a **loud** failure. Check freshness for the quiet ones.
5. Write a **last day** into every collector.